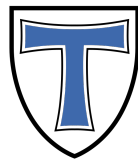


JUSTUS-LIEBIG-



UNIVERSITÄT
GIESSEN

GRUPPENNUMMER: 308

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Übrige Gruppenmitglieder:

Florian Adamczyk

Björn Becker

Felix Neumann (Sprecherin)

Kolja Scriba

Protokoll von:

Felix Hollfoth

felix.hollfoth@stud.uni-giessen.de

Matrikel Nr.: 8221324

Modul: Künstliche Intelligenz I
Dozent: Prof. Dr. Christian Heiliger
Semester: Wintersemester 2025/26

Inhaltsverzeichnis

1 Logbuch

2 Zusammenfassung des Logbuchs

2.1 Ausgangssituation

Mit dem Ziel ein neuronales Netz basierenden auf dem California Housing Datensatz von Scikit Learn zu entwickeln, wird Tensorflow bzw. Keras zum Modelltraining verwendet.

Hilfsfunktionen sind bereits programmiert, wodurch der Datensatz eingelesen und bereinigt wird. Ebenfalls liegen Funktionen zum generieren einheitlicher Datensplits und für einheitliche Metriken zum Ergebnisvergleich vor. Ein erstes Modell zur Einschätzung des neuronalen Netzes gegenüber der linearen Regression liegt bereits vor. (**florians_code**)

Meine zentrale Aufgabe lautet nun dieses Netz bezüglich ausgewählter Hyperparameter zu Verbessern und deren Einfluss auf das Lernen des Netzes zu untersuchen.

2.2 Variation von Hyperparametern am neuronalen Netz

Aus vorherigen Analysen (**florians_code**) geht hervor, dass den Daten ein nicht-linearer Zusammenhang zur Grunde liegt, weshalb die lineare Regression Interaktionen zwischen Features nicht vollständig darstellen kann. Um nichtlineare Effekte darstellen zu können werden nichtlineare Aktivierungsfunktionen implementiert. Dabei werden die gängigsten Aktivierungsfunktionen (**aktivierungsfunktionen2024**) (**selu2025**) getestet.

Ob eine Aktivierungsfunktion stabile Gradienten im Backpropagation-Prozess erzeugen kann hängt ebenfalls von der Größenordnung der Eingabedaten ab. Untersucht werden in diesem Zusammenhang die originalen, die standardisierten und die zwischen null und eins skalierten Daten. Die Netzstruktur wird von dem bereits trainierten Modell übernommen (**florians_code**).

Tabelle 1: Vergleich des Bestimmtheitsmaßes der Testdaten R^2 je nach Aktivierungsfunktion und Datentransformation

Aktivierungsfunktion	Datentransformation		
	keine	standard	min0-max1
relu	-0,1504	0,7561	0,7618
tanh	0,309	0,7668	0,7405
sigmoid	0,5689	0,7529	0,6786
elu	0,559	0,7479	0,7165
selu	0,2315	0,7577	0,7462
leaky relu	-0,5892	0,7564	0,7541

Wie erwartet führen die unterschiedlichen Intervalle der Featuredaten dazu, dass ein Lernen ohne Datentransformation nicht geeignet ist (siehe Tabelle ??). Insgesamt zeigt sich, dass die Aktivierungsfunktionen bei standardisierten Daten durch die gut

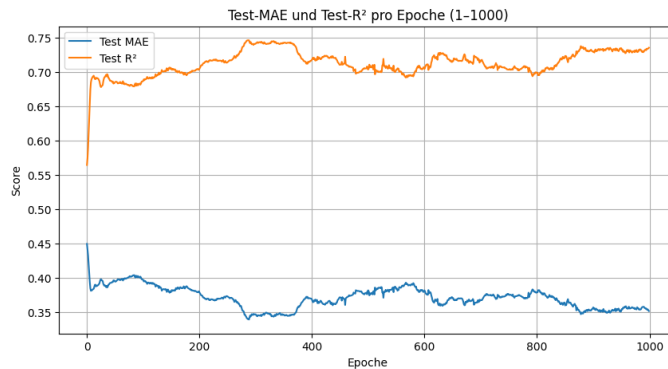


Abbildung 1: Verlauf des MAE in \$100.000 und des R^2 über den Epochenverlauf

konditionierten Bereiche und nur zwei hidden Layers einen geringen Einfluss auf das Lernverhalten aufweisen.

Ähnliches Verhalten kann auch bei Optimierung von Lernrate, Batch-Size und Epochenanzahl beobachtet werden. Neben einer groben Einordnung mögliche Wertebereiche:

1. Lernrate: [0.001, 0.01]
2. Batch-Size: [16, 32, 64, 128]
3. Epochenanzahl: [200, 500]

konnte vor allem folgendes festgestellt werden. Die Lernrate und Batch-Size schwanken in dem oben angegeben Intervall um bis zu 3 % im R^2 Testscore, was auf ein zufälliges Training hindeutet und im Folgenden durch die Varianz bei erneutem Ausführen des selben Codes zurückzuführen ist. Wie in Abbildung ?? zu erkennen ist und durch mehrere Durchläufe bestätigt wurde, sind 100 Epochen noch nicht ausreichend um ein lokales Minimum in der Fehlerlandschaft zu erkennen. Im Vergleich muss auch betrachtet, dass das Overfitting ab 500 Epochen deutlich zunimmt und dann mehr Rauschen wie Informationen gelernt werden. Auch die Anwendung einer Anpassung der Lernrate während des Trainings (LR-Schedule) führt nicht zu einem verbesserten Lernen.

Wie bereits erwähnt ist während dem Training aufgefallen, dass ein erneutes Lernen eines Netzes zum Teil zu verschiedenen Ergebnissen führen kann. Dies ist problematisch um eine Verbesserung mittels Hyperparameteroptimierung zu erreichen, wenn die Varianz bereits durch eine erneute Ausführung gegeben ist. Abbildung ?? zeigt die statistische Verteilung des erreichten Testscores über 100 Modelle (**reproduzierbarkeit**). Daraus resultiert ein Mittelwert von 0.7481 mit einer Standardabweichung von 0.0124. Dadurch zeigt sich das vermutete Problem, dass die

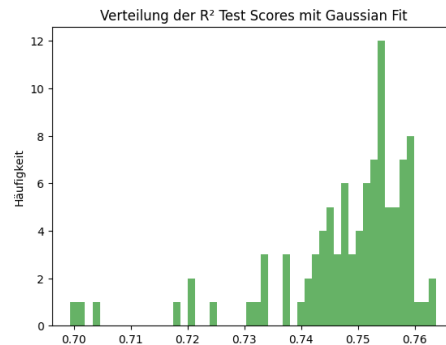


Abbildung 2: Verteilung des erreichten R^2 Testscores über 100 Modelle

Hyperparameter sich nicht weiter optimieren lassen, da Sie durch die Varianz des Modells überdeckt werden. Gründe dafür sind andere Initialgewichte in Keras, unterschiedliche Ausführungsreihenfolgen auf der CPU und die Reproduzierbarkeit von Floating-Points.

2.3 Random Search

Da bisher nur Hyperparameter einzeln betrachtet wurden, erfolgt ein systematischer Test mittels Random Search. Dies wird Grid Search vorgezogen aufgrund der zu erwarteten Rechenzeit und Ergebnisse der Varianten (**randomsearch2012**). Über 100 Modelle zeigte sich, dass ein R^2 Testscore von 0.7759 erreicht werden kann. Durch eine Betrachtung der besten zehn Modellen zeigte sich, dass kein Hyperparameter verhältnismäßig oft für ein gutes Ergebnis sorgt. Dies bestätigt den hohen Zufall, der momentan im Training herrscht. Daher werden in Zukunft immer fünf Mal ein Modell mit identischen Parametern trainiert. Ein weiteres Problem war, dass weitere Hyperparameter von Kolja parallel bearbeitet wurde, aber durch Git-Probleme ich nicht mit diesen Erkenntnissen arbeiten konnte.

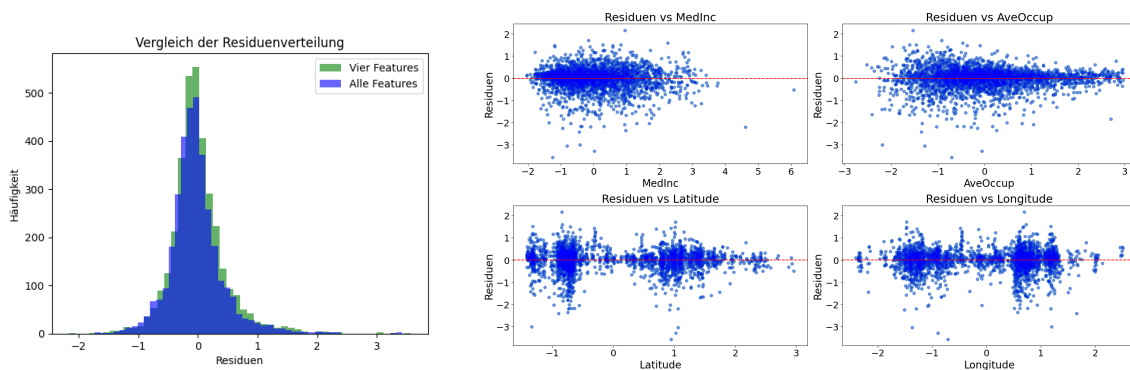
2.4 Featureselektion des Modells und Ensemble Averaging

An dieser Stelle erfolgte eine Betrachtung der bisher erreichten Ergebnisse, um neue Ideen für weitere Verbesserungen zu erlangen. Bei der Analyse viel auf, dass hauptsächlich nur vier Features (MedInc, AveOccup, Longitude und Latitude) für den Hauspreis wichtig sind. Daher wurde ein neues Modell mit den selektierten Features trainiert.

Es zeigt sich, dass der Score sich nicht verschlechtert, wenn das Modell nur auf vier Features trainiert wird. Somit enthalten die anderen Features keine zusätzlichen Informationen bzw. enthalten Sie genauso viel Rauschen wie Informationen.

Betrachtet man das Residuen-Histogramm für vier und allen acht Features, sieht man eine gleichmäßige Verteilung um die Null herum. Ebenfalls erhalten wir eine schlankere Verteilung, welches auf eine bessere Vorhersage bzw. einen besseren Score schließen lässt (siehe Abbildung ??).

Durch die geringeren Freiheitsgrade wird Overfitting reduziert, kein unnötiges Rauschen erlernt, und das Training vereinfacht. Die Ergebnisse lassen darauf schließen, dass die Modellleistung durch die Datenqualität anstatt durch die Modellkomplexität begrenzt ist. Das zeigt sich auch in der Abbildung ??, da keine Cluster, unterschiedliche Ordnungen und kein Bias erkennbar sind (**residuenanalysis2022**).



(a) Residuenhistogramme für acht und vier Features

(b) Residuen für jedes der vier ausschlaggebenden Features

Abbildung 3: Vergleich der Residuenplots bei unterschiedlicher Featureanzahl

Mittels Ensemble Averaging (**ensemblewiki2026**) sollen die Vorteile der bisher betrachteten Module zusammengefasst werden. Aus einer Kombination von vier Modellen mit jeweils unterschiedlichen Hyperparametern und Featureanzahl kann ein R^2 Testscore von 0.7983 erreicht werden. (**ensemble_code**) Dadurch zeigt sich deutlich, dass die Modelle nicht Redundant sind und exakt korrelieren.

2.5 Zwischenstand und Ausblick

- Random Search über alle Parameter - Hyperparameter nicht weiter untersucht werden - Komplexität ausreichend - Dropout, Netzstabilisierung noch testen - Ensemble ist gut für Verbesserung - Ausreißer anschauen eventuell auf datenbereinigung zurückzuführen

3 Einschätzung der Gruppenmitglieder

Björn Becker:

Positiv:

Negativ:

Felix Neumann:

Positiv:

Negativ:

Florian Adamczyk:

Positiv:

Negativ:

Kolja Scriba:

Positiv:

Negativ:

4 Quellen und Hilfsmittel